



**PROJET D'INTERVENTION
MAÎTRISE EN INFORMATIQUE**

UNIVERSITÉ DU QUÉBEC À RIMOUSKI

DÉPARTEMENT DE MATHÉMATIQUES, D'INFORMATIQUE ET DE GÉNIE

RAPPORT

**Plateforme multi-agents IA pour la supervision
intelligente d'un réseau IoT simulé**

ÉQUIPE :

Socé Ndiaye

SUPERVISEUR :

Yacine Yaddaden, Ph. D.

Date : 1^{er} août 2026

Table des matières

1	Introduction	3
2	Objectif	3
3	Méthodologie	3
4	Installation des outils	4
4.1	Docker Desktop	4
4.2	Mosquitto (Broker MQTT)	5
4.3	n8n	5
4.4	PicoClaw	5
4.5	Python 3.13 et bibliothèques	6
5	Architecture globale	6
6	Fonctionnalités implémentées	6
6.1	Simulation IoT	6
6.2	Agent de surveillance (MonitorAgent)	7
6.3	Agent d'analyse (AnalysisAgent) via PicoClaw	7
6.4	Agent de décision (DecisionAgent)	8
6.5	Orchestration n8n	8
6.5.1	Workflow alert	8
6.5.2	Workflow fire	9
6.5.3	Workflow hvac	9
6.5.4	Workflow power	10
6.6	Dashboard Streamlit	10
6.6.1	Notifications et données capteurs	11
6.6.2	Contrôle interactif des capteurs	12
6.6.3	Création dynamique de zones	13
6.6.4	Chiffres clés et activité	15
7	Défis rencontrés et solutions adoptées	16
7.1	Performance des modèles locaux	16
7.2	Souveraineté et confidentialité des données	16
7.3	Filtrage de sévérité	16
7.4	Synchronisation Streamlit et MQTT	17
7.5	Configuration de PicoClaw	17
8	Résultats et validation	17

9 Conclusion	19
9.1 Objectifs atteints	19
9.2 Apports techniques	19
9.3 Perspectives	19
9.4 Impact et utilisation	19

1 Introduction

Ce rapport présente la conception et la réalisation d'une plateforme multi-agents basée sur l'intelligence artificielle pour la supervision intelligente d'un réseau IoT simulé dans un contexte de bâtiment intelligent (smart building). Le projet s'inscrit dans le cadre du projet d'intervention de la Maîtrise en informatique. L'objectif principal était de développer un système autonome capable de détecter les anomalies remontées par des capteurs IoT en temps réel, de les analyser à l'aide d'un modèle de langage avancé, et de déclencher automatiquement les actions correctives ainsi que les notifications appropriées.

Ce travail combine plusieurs technologies modernes : Docker pour l'orchestration des services, Mosquitto comme broker MQTT pour la communication temps réel, PicoClaw associé à l'API Anthropic (Claude Sonnet 4.6) pour l'analyse par LLM, n8n pour l'automatisation des workflows, Streamlit pour le tableau de bord, et Gmail SMTP pour les notifications au responsable du bâtiment.

2 Objectif

L'objectif de ce projet est de concevoir une architecture multi-agents complète permettant de superviser un réseau IoT simulé de manière intelligente et autonome. Plus spécifiquement, le projet vise à :

- Simuler un réseau de capteurs ESP32 virtuels distribués dans plusieurs zones d'un bâtiment
- Détecter les anomalies en temps réel grâce à un agent de surveillance dédié
- Analyser les situations critiques à l'aide d'un LLM (Claude Sonnet 4.6 via PicoClaw)
- Décider et déclencher automatiquement des actions via des workflows n8n
- Notifier le responsable du bâtiment par courriel pour chaque incident
- Offrir un tableau de bord temps réel pour la supervision interactive
- Permettre la création dynamique de nouvelles zones directement depuis l'interface

Le système développé doit être capable de recevoir des lectures MQTT en continu, filtrer les anomalies critiques, produire un diagnostic structuré, déclencher les actions correspondantes et envoyer les alertes aux personnes concernées, le tout dans une architecture entièrement conteneurisée.

3 Méthodologie

Le projet a été réalisé selon une approche itérative découpée en cinq phases progressives. Chaque phase produit un livrable fonctionnel, testable indépendamment, avec des retours réguliers du superviseur. La figure 1 illustre la méthodologie adoptée.



FIGURE 1 – Méthodologie du projet — approche itérative en cinq phases

Cette méthodologie repose sur trois principes :

- **Développement itératif** : chaque phase produit un livrable fonctionnel testable indépendamment
- **Évolution technologique** : la stack a été migrée d'Ollama local vers PicoClaw + Claude Sonnet 4.6 pour améliorer la qualité et la vitesse d'analyse
- **Validation continue** : des tests de scénarios contrôlés sont menés à chaque étape via injection MQTT directe

4 Installation des outils

Cette section détaille l'installation et la configuration de tous les outils nécessaires au projet.

4.1 Docker Desktop

Docker Desktop constitue la base de l'infrastructure et permet le déploiement des services dans des conteneurs isolés. L'installation s'est effectuée via le site officiel de Docker. Quatre conteneurs cohabitent dans le projet : Mosquitto (broker MQTT), n8n (workflows), PicoClaw (agent LLM) et le conteneur applicatif smart-building-multiagent. La figure 2 montre les conteneurs actifs.

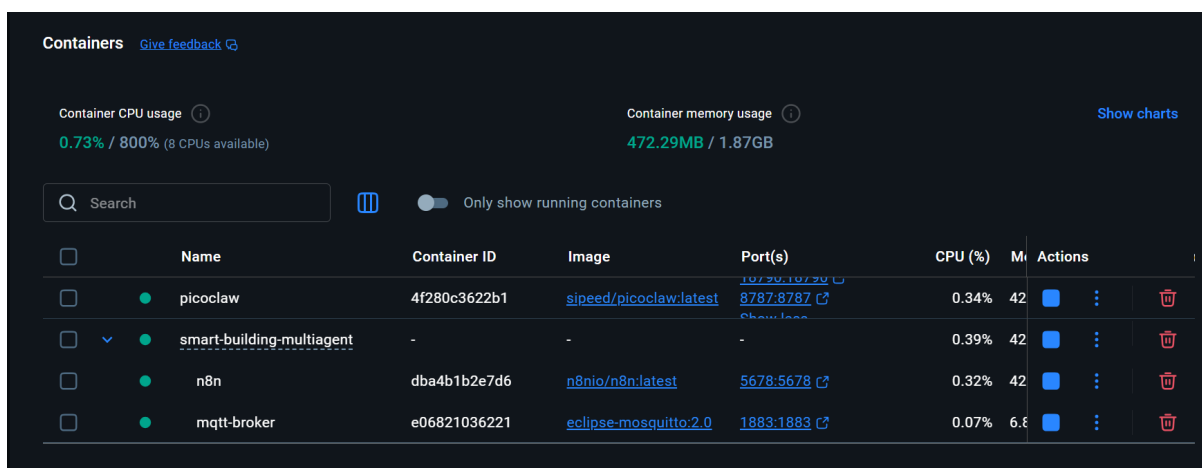


FIGURE 2 – Conteneurs Docker actifs dans le projet

4.2 Mosquitto (Broker MQTT)

Mosquitto a été déployé via Docker Compose comme broker MQTT léger et performant. Il constitue le nerf central de la communication entre les capteurs simulés et les agents.

```
docker run -d --name mqtt-broker -p 1883:1883 eclipse-mosquitto:2.0
```

Les topics utilisés suivent une hiérarchie claire : building/{zone} pour les mesures capteurs, status/{zone} pour l'état des capteurs, control/{zone} pour les commandes de contrôle et decisions/{zone} pour les décisions de l'IA publiées vers le dashboard.

4.3 n8n

n8n a été installé via Docker pour orchestrer les workflows d'action déclenchés par le DecisionAgent. Cette plateforme d'automatisation permet de créer visuellement des workflows qui reçoivent des webhooks, appliquent des transformations et déclenchent des envois de courriels.

```
docker run -d --name n8n -p 5678:5678 n8nio/n8n:latest
```

4.4 PicoClaw

PicoClaw est un framework agent ultra-léger écrit en Go qui sert d'interface entre le AnalysisAgent et l'API Anthropic. Il fonctionne avec moins de 10 Mo de RAM et démarre en moins d'une seconde.

```
docker run -d --name picoclaw -p 18790:18790
-v C:\picoclaw:/root/.picoclaw ghcr.io/sipeed/picoclaw:latest
```

La configuration se fait via le fichier `config.json` monté en volume et pointe vers le modèle `claude-sonnet-4-6` avec une clé API Anthropic.

4.5 Python 3.13 et bibliothèques

Les simulateurs, les agents et le dashboard sont écrits en Python 3.13. Les principales bibliothèques utilisées sont :

- **paho-mqtt** : client MQTT pour la publication/souscription
- **streamlit** : framework pour le dashboard web
- **plotly** : graphiques interactifs
- **requests** : appels HTTP vers les webhooks n8n

5 Architecture globale

L'architecture du système est structurée en cinq couches complémentaires. La couche 1 correspond à la simulation IoT avec quatre zones (Étage 1, Étage 2, Étage 3, Salle serveur) et huit capteurs virtuels au minimum. La couche 2 est le broker MQTT Mosquitto qui reçoit et distribue tous les messages. La couche 3 regroupe les trois agents IA (Monitor, Analysis, Decision). La couche 4 est l'orchestration via n8n avec quatre workflows spécialisés. La couche 5 est la notification finale par courriel et l'affichage sur le dashboard Streamlit.

Le flux complet est le suivant : *ESP32 simulé* → *MQTT* → *MonitorAgent* → *AnalysisAgent* (*PicoClaw* + *Claude*) → *DecisionAgent* → *n8n Webhooks* → *Courriel* + *Dashboard*.

6 Fonctionnalités implémentées

Cette section présente en détail les fonctionnalités développées sur chaque composante du système.

6.1 Simulation IoT

Les capteurs sont simulés par des scripts Python héritant tous d'une classe abstraite `BaseSensor` qui gère la connexion MQTT, la publication cyclique, le contrôle démarrage/pause via MQTT et l'injection aléatoire d'anomalies. Chaque zone possède ses propres types de mesures :

- **Étage 1** : température, humidité, CO₂
- **Étage 2** : température, mouvement, luminosité
- **Étage 3** : température, humidité, fumée
- **Salle serveur** : température, charge CPU, puissance électrique

Les capteurs publient leurs mesures toutes les 5 secondes sur le topic `building/{zone}`. Une probabilité de 5% d'injection d'anomalie a été fixée pour simuler des événements réalistes.

6.2 Agent de surveillance (MonitorAgent)

Le MonitorAgent souscrit à tous les topics `building/#` et compare les valeurs reçues à des seuils prédéfinis. Il classe chaque anomalie en trois niveaux de sévérité : `medium`, `high`, `critical`. Les règles de sévérité tiennent compte du contexte de la zone : la fumée détectée en Étage 3 est immédiatement classée `critical`, tandis qu'une simple élévation de CO₂ en Étage 1 reste `medium`.

6.3 Agent d'analyse (AnalysisAgent) via PicoClaw

Le AnalysisAgent reçoit les anomalies `high` et `critical` et les transmet à PicoClaw via une commande `docker exec`. Le prompt construit demande à Claude Sonnet 4.6 de fournir un diagnostic structuré en JSON contenant : `diagnostic`, `cause_probable`, `risque`, `action_recommandee` et `urgence`.

```

PICOCLAW
02:46:17 WRN tools home/runner/work/picoclaw/picoclaw/pkg/tools/shell.go:180 > deny patterns
are disabled, all commands will be allowed

❤️ Bonjour ! ❤️ Je suis **PicoClaw**, votre assistant IA personnel !

---

### 🤖 Qui suis-je ?
Je suis un assistant IA ultra-léger écrit en Go, conçu pour tourner sur du matériel modeste a
vec moins de 10MB de RAM. Je suis actuellement déployé dans votre environnement Docker et con
necté à votre projet **Smart Building** !

---

### 📋 Ce que je fais déjà pour vous
Depuis notre conversation, je :
- ✅ Analyse les **anomalies IoT** en temps réel de vos capteurs (`esp32-server-room`, `esp32
-floor3`)
- ✅ Détecte les situations critiques : **incendies**, **surchauffes**, **surcharges CPU**, *
anomalies électriques**
- ✅ Réponds en **JSON structuré** avec diagnostic, risque, cause et action recommandée

```

FIGURE 3 – PicoClaw connecté à l'API Anthropic, opérationnel pour l'analyse d'anomalies

La figure 3 illustre PicoClaw en fonctionnement, capable d'analyser en temps réel les

anomalies remontées par les capteurs ESP32. L'intégration a permis un gain de performance significatif : le temps de réponse est passé d'environ 90 secondes avec Ollama local (Mistral 7B) à environ 7 secondes avec Claude Sonnet 4.6, soit une accélération d'un facteur 13.

6.4 Agent de décision (DecisionAgent)

Le DecisionAgent reçoit le diagnostic et évalue une liste ordonnée de règles pour déterminer quelles actions déclencher. Les règles sont :

1. `floor3 + smoke=1 → action fire`
2. `floor3 + risque critical/high + urgence → action fire`
3. `server_room + risque critical/high + urgence → actions power + hvac`
4. `Autres zones + risque critical/high + urgence → action hvac`
5. `risque medium → action alert`

Chaque action correspond à un webhook n8n. Le DecisionAgent publie également chaque décision sur le topic `decisions/{zone}` pour que le dashboard puisse l'afficher.

6.5 Orchestration n8n

Quatre workflows ont été implémentés dans n8n, chacun spécialisé pour un type d'action. Chaque workflow suit une structure similaire : réception d'un webhook, transformation JavaScript des données, puis envoi d'un courriel via SMTP Gmail.

6.5.1 Workflow alert

Ce workflow gère les alertes standard pour les anomalies de sévérité `medium`. Il conditionne l'envoi via un nœud If et envoie une notification simple par courriel.

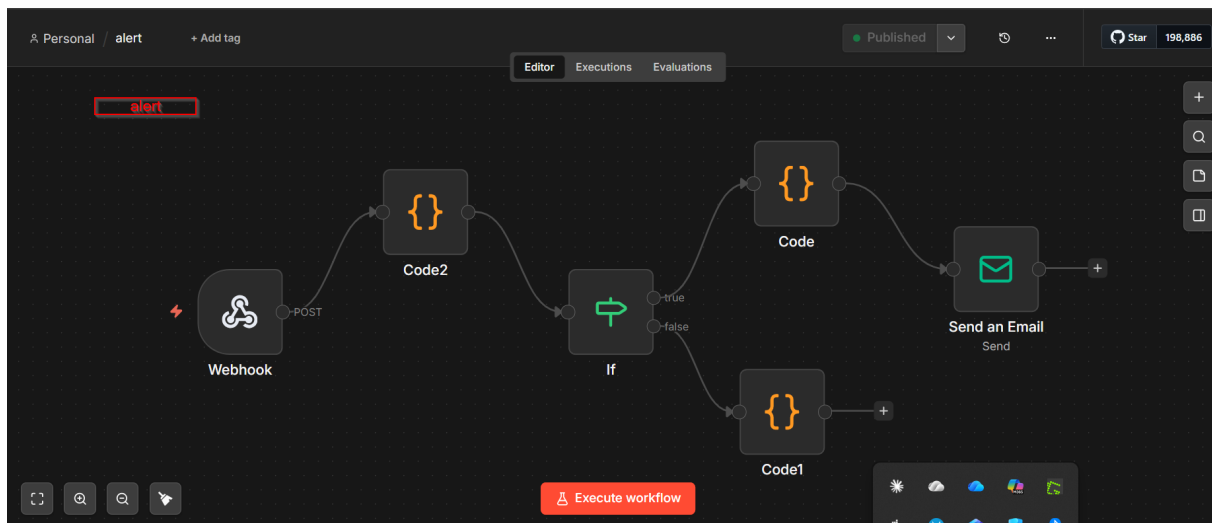


FIGURE 4 – Workflow n8n alert avec branchement conditionnel

6.5.2 Workflow fire

Ce workflow est déclenché en cas d'incendie détecté et envoie une alerte urgente avec les procédures d'évacuation et le numéro des pompiers.

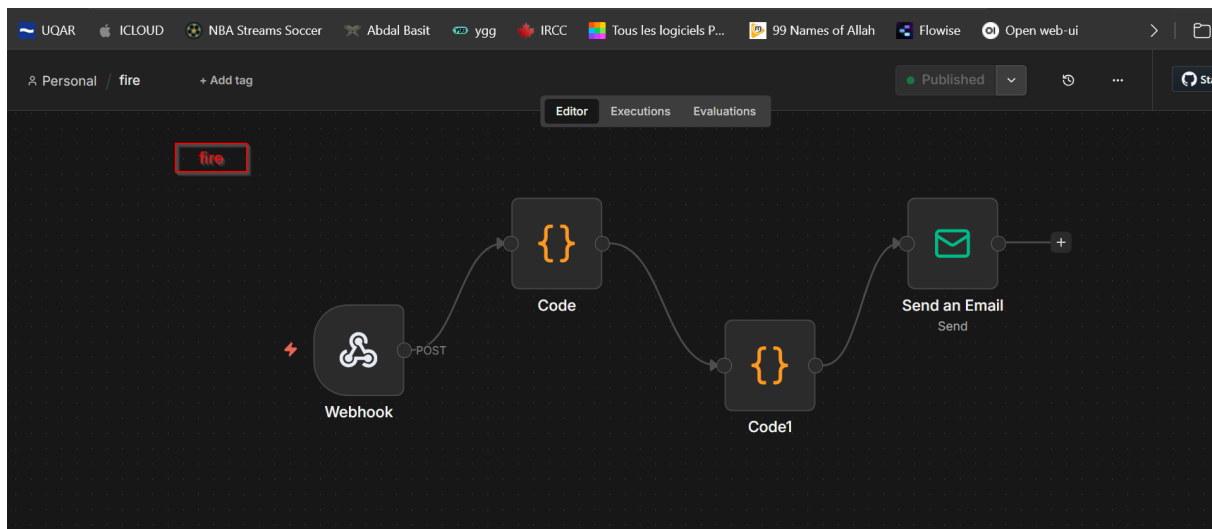


FIGURE 5 – Workflow n8n fire pour les alertes d'incendie

6.5.3 Workflow hvac

Ce workflow gère les alertes de régulation ventilation/climatisation, principalement pour les surchauffes détectées dans les étages ou la salle serveur.

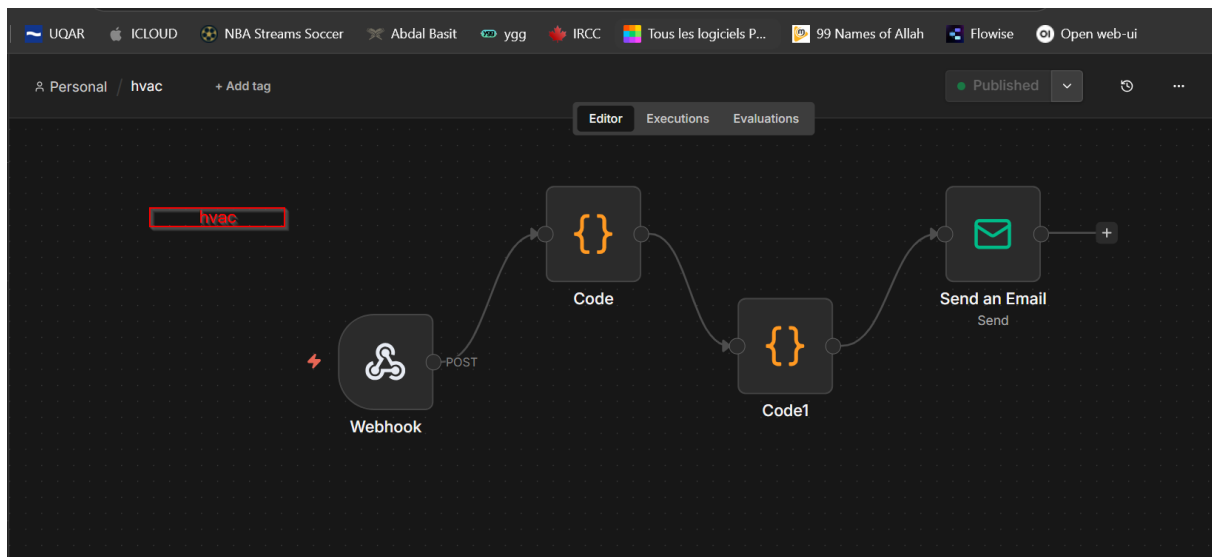


FIGURE 6 – Workflow n8n hvac pour la régulation thermique

6.5.4 Workflow power

Ce workflow est spécifique à la salle serveur et permet de notifier le service électrique en cas de surcharge critique.

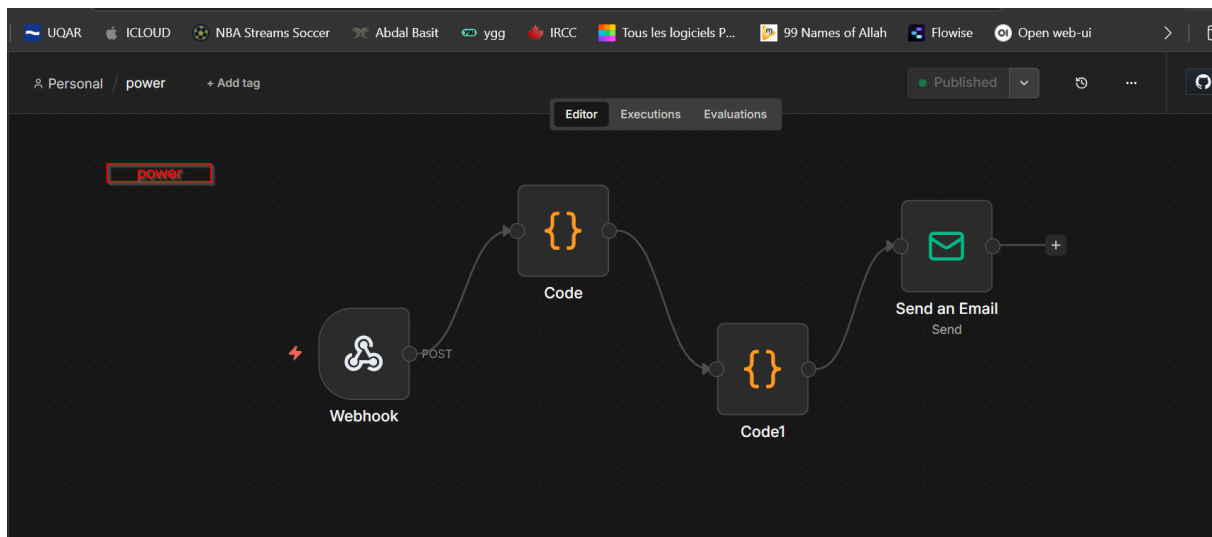


FIGURE 7 – Workflow n8n power pour la gestion électrique

6.6 Dashboard Streamlit

Un tableau de bord web a été développé avec Streamlit pour offrir une supervision temps réel et interactive du système. Le dashboard souscrit aux topics MQTT et affiche

en direct l'état de toutes les zones, les anomalies détectées et les décisions de l'IA. La figure 8 présente la vue globale du dashboard.

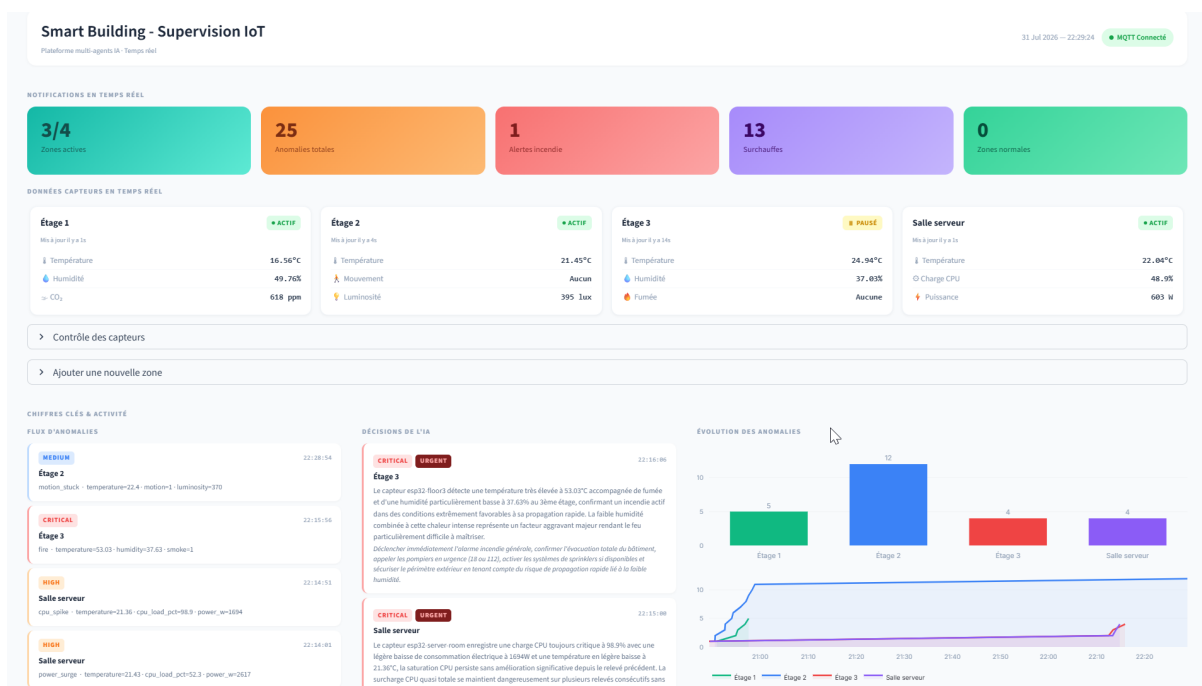


FIGURE 8 – Vue générale du dashboard Streamlit — supervision temps réel

Le dashboard est structuré en plusieurs sections indépendantes qui se complètent pour offrir une vision d'ensemble et un contrôle interactif complet.

6.6.1 Notifications et données capteurs

La partie supérieure du dashboard regroupe cinq indicateurs de performance clés (zones actives, anomalies totales, alertes incendie, surchauffes, zones normales) ainsi que les cartes des capteurs en temps réel. La figure 9 montre cette section.

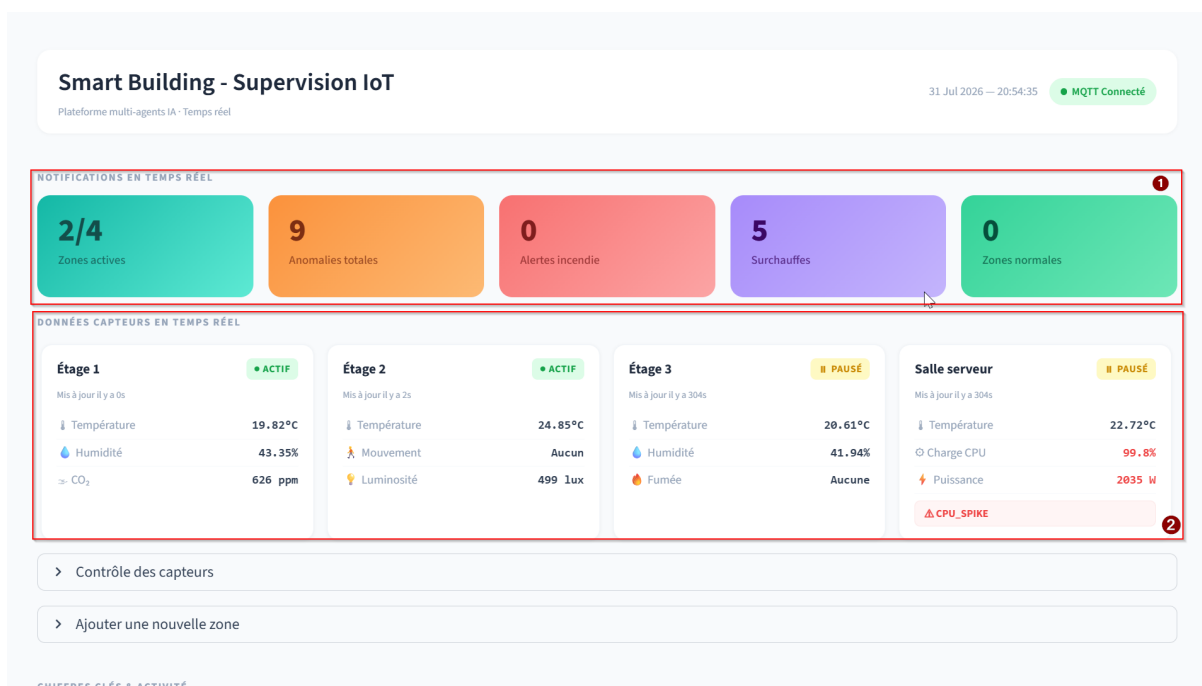


FIGURE 9 – Section supérieure — KPI et cartes des capteurs

Chaque carte capteur affiche les valeurs actuelles avec code couleur (rouge si dépassement de seuil), le statut du capteur, et la dernière mise à jour. Une alerte visuelle apparaît lorsqu'une anomalie est détectée (par exemple CPU_SPIKE sur la Salle serveur).

6.6.2 Contrôle interactif des capteurs

Un panneau repliable permet de démarrer, arrêter ou reprendre chaque capteur individuellement via des boutons directement dans l'interface. Les commandes sont publiées sur le topic MQTT `control/{zone}` et sont interceptées par les simulateurs.

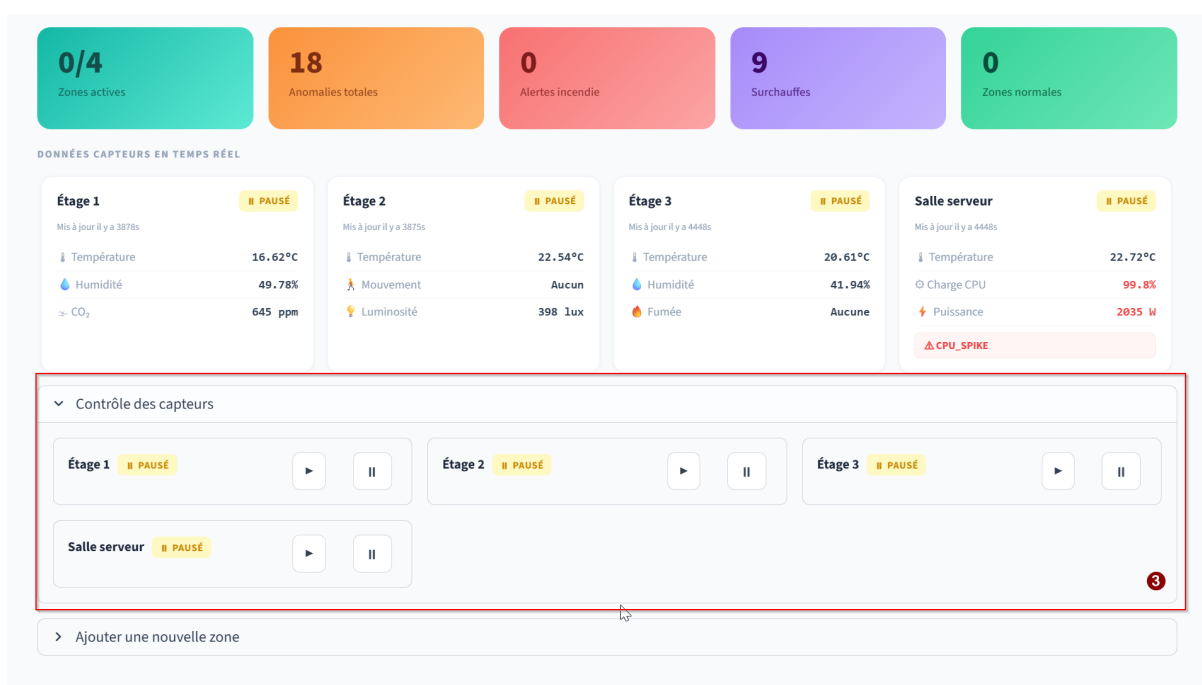


FIGURE 10 – Contrôle interactif des capteurs depuis le dashboard

6.6.3 Création dynamique de zones

Une fonctionnalité de création à chaud de nouvelles zones a été implémentée. L'utilisateur peut définir un nom, un identifiant et sélectionner les types de capteurs voulus (température, humidité, CO₂, luminosité, mouvement, fumée, charge CPU, puissance). La figure 11 présente le formulaire.

The screenshot shows a dashboard with five status cards at the top: 'Zones actives' (green), 'Anomalies totales' (orange), 'Alertes incendie' (red), 'Surchauffes' (purple), and 'Zones normales' (green). Below these is a section titled 'DONNÉES CAPTEURS EN TEMPS RÉEL' containing four panels for 'Étage 1', 'Étage 2', 'Étage 3', and 'Salle serveur'. Each panel displays various sensor readings like temperature, humidity, movement, and CO2. Below the sensor panels is a 'Contrôle des capteurs' section with a sub-section 'Ajouter une nouvelle zone'. This section contains a form with two input fields: 'Nom de la zone' (with example 'Cafétéria') and 'Identifiant (sans espaces)' (with example 'cafeteria'). There is also a 'Types de capteurs' dropdown menu currently showing 'temperature' and 'humidity'. A 'Créer la zone' button is at the bottom of the form. A red box highlights the entire form area, and a red circle with the number 4 is in the bottom right corner of the dashboard area.

FIGURE 11 – Formulaire de création dynamique de zone

À la validation, une instance de `GenericSensor` est créée et démarrée, la zone est enregistrée dans l'état partagé du dashboard, et elle apparaît immédiatement dans la liste des zones supervisées (figure 12).

The screenshot shows the same dashboard as Figure 11, but now the 'Cafeteria' zone has been added to the list of supervised zones. It is shown as a new panel on the right, with a green 'ACTIF' status. The 'Cafeteria' panel displays sensor readings: Temperature 20.98°C, Humidity 45.05%, and Smoke 'Aucune'. A red box highlights the 'Cafeteria' panel, and a red arrow points from the 'Ajouter une nouvelle zone' form in Figure 11 to this panel. A red circle with the number 9 is in the bottom right corner of the dashboard area.

FIGURE 12 – Nouvelle zone Cafétéria intégrée aux zones supervisées

Le formulaire prend en charge tous les types de capteurs configurables. La figure 13 montre le choix des capteurs pour une nouvelle zone.

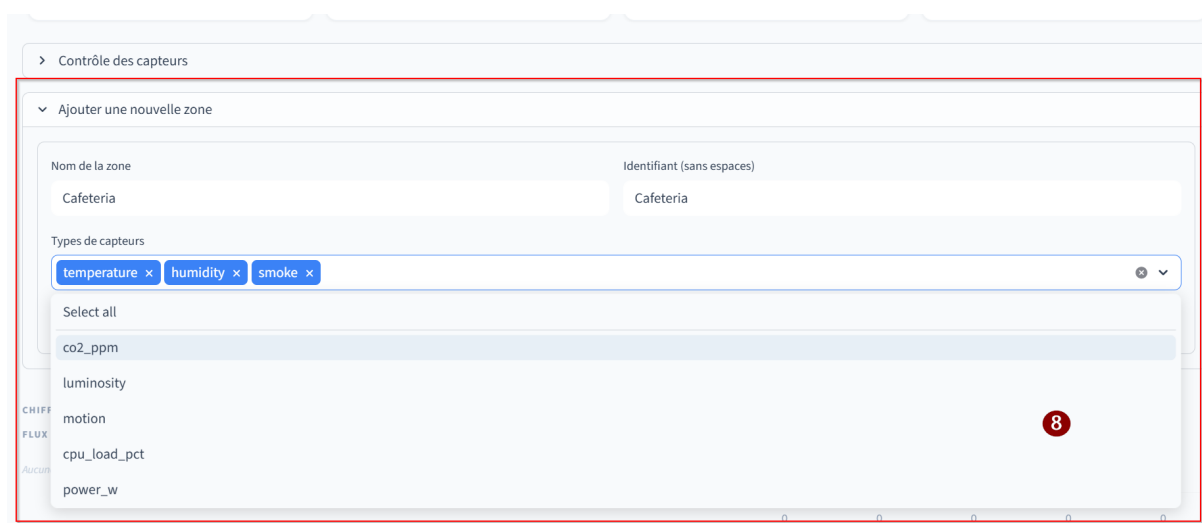


FIGURE 13 – Sélection des types de capteurs lors de la création

6.6.4 Chiffres clés et activité

La partie inférieure du dashboard présente trois panneaux synchronisés : le flux d'anomalies détectées, les décisions de l'IA générées par Claude Sonnet, et les graphiques d'évolution des anomalies par zone.

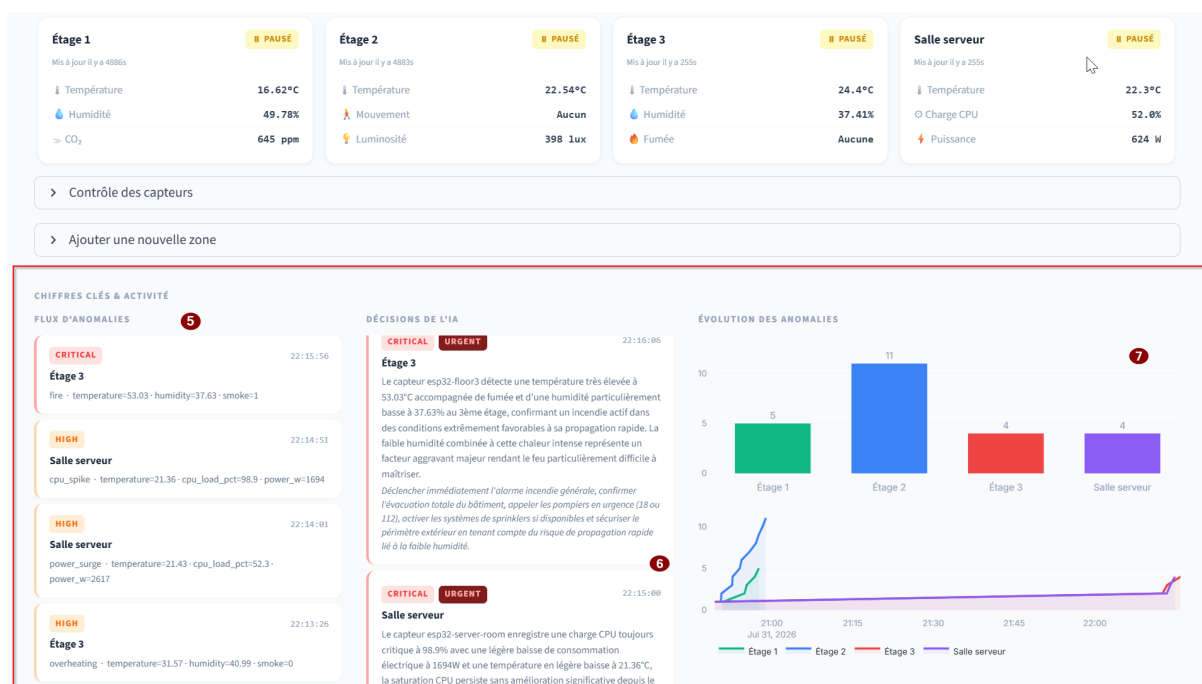


FIGURE 14 – Panneaux du bas — flux d'anomalies, décisions IA, graphiques

Les décisions affichent le niveau de risque, le badge d'urgence, le diagnostic complet et l'action recommandée générés par Claude. Les graphiques Plotly montrent le total d'anomalies par zone (barres) et l'évolution cumulative dans le temps (courbes).

7 Défis rencontrés et solutions adoptées

Le projet a soulevé plusieurs défis techniques dont les principaux sont détaillés ci-dessous.

7.1 Performance des modèles locaux

La première version du système utilisait Ollama avec le modèle Mistral 7B en local. Bien que fonctionnel, ce choix présentait des limitations importantes :

- Temps de réponse d'environ 90 secondes par analyse
- Consommation mémoire élevée
- Diagnostics parfois génériques

La solution retenue a été la migration vers PicoClaw associé à l'API Anthropic (Claude Sonnet 4.6). Ce changement a permis :

- Une réduction du temps d'analyse à environ 7 secondes (facteur 13)
- Une qualité de diagnostic supérieure avec des actions concrètes chiffrées
- Une consommation locale minimale (moins de 10 Mo pour PicoClaw)

7.2 Souveraineté et confidentialité des données

Le passage à un LLM cloud soulève une question importante de vie privée : les prompts contenant les données des capteurs transitent par les serveurs d'Anthropic. Dans un contexte réel de smart building, les capteurs de mouvement, de présence ou de CO₂ peuvent révéler des informations sensibles sur l'occupation et les habitudes des personnes. Ce compromis est présenté comme un choix architectural conscient entre performance et souveraineté, avec des pistes d'évolution proposées (utilisation d'Ollama en local, anonymisation des prompts, déploiement de Claude via Anthropic Enterprise).

7.3 Filtrage de sévérité

Pour éviter la saturation du LLM par des anomalies de faible criticité, un filtre a été implémenté dans l'orchestrateur : seules les anomalies `high` et `critical` sont transmises à Claude, tandis que les anomalies `medium` déclenchent uniquement le workflow alert sans analyse LLM. Cette décision architecturale est présentée comme un mécanisme de triage pour optimiser l'utilisation du modèle.

7.4 Synchronisation Streamlit et MQTT

Streamlit réexécute intégralement le script Python à chaque interaction ou rafraîchissement automatique, ce qui pose problème pour maintenir une connexion MQTT persistante et l'état des capteurs. La solution retenue utilise le décorateur `@st.cache_resource` pour instancier l'état partagé et le client MQTT une seule fois, combiné à un verrou `threading.Lock` pour synchroniser les accès entre le thread MQTT et le thread principal de Streamlit.

7.5 Configuration de PicoClaw

La configuration initiale de PicoClaw a été délicate en raison des changements de format entre versions et de la documentation encore incomplète. Plusieurs itérations ont été nécessaires pour identifier le bon format des champs `provider`, `model`, `api_keys` et pour retirer le champ `api_base` qui provoquait des erreurs. La configuration finale utilise le modèle `claude-sonnet-4-6` avec la clé API stockée directement dans le fichier `config.json`.

8 Résultats et validation

Quatre scénarios de tests ont été définis et validés par injection MQTT contrôlée. Le tableau 1 présente une synthèse des résultats.

#	Scénario	Valeurs injectées	Actions déclenchées
S1	Incendie étage 3	smoke=1, temp=62.5°C	fire + courriel urgent
S2	Surchauffe salle serveur	temp=48°C, CPU=97.5%, power=2800W	power + hvac + courriels
S3	Qualité d'air dégradée	CO ₂ =2200 ppm	alert + courriel standard
S4	Panne éclairage	luminosity=0, motion=1	alert + courriel standard

TABLE 1 – Résultats des scénarios de tests

Les quatre scénarios ont été validés avec succès. La latence moyenne mesurée entre la détection d'une anomalie et la réception du courriel est inférieure à 10 secondes, ce qui démontre la réactivité de l'ensemble de la chaîne.

La figure 15 montre un exemple de courriel réellement reçu sur téléphone mobile lors du scénario S1 (incendie étage 3). Le contenu est entièrement généré par le pipeline : le diagnostic est produit par Claude Sonnet 4.6 à partir des valeurs capteurs, l'action recommandée est contextualisée avec le numéro d'urgence, et la mise en forme HTML est réalisée dans le workflow `n8n fire`.

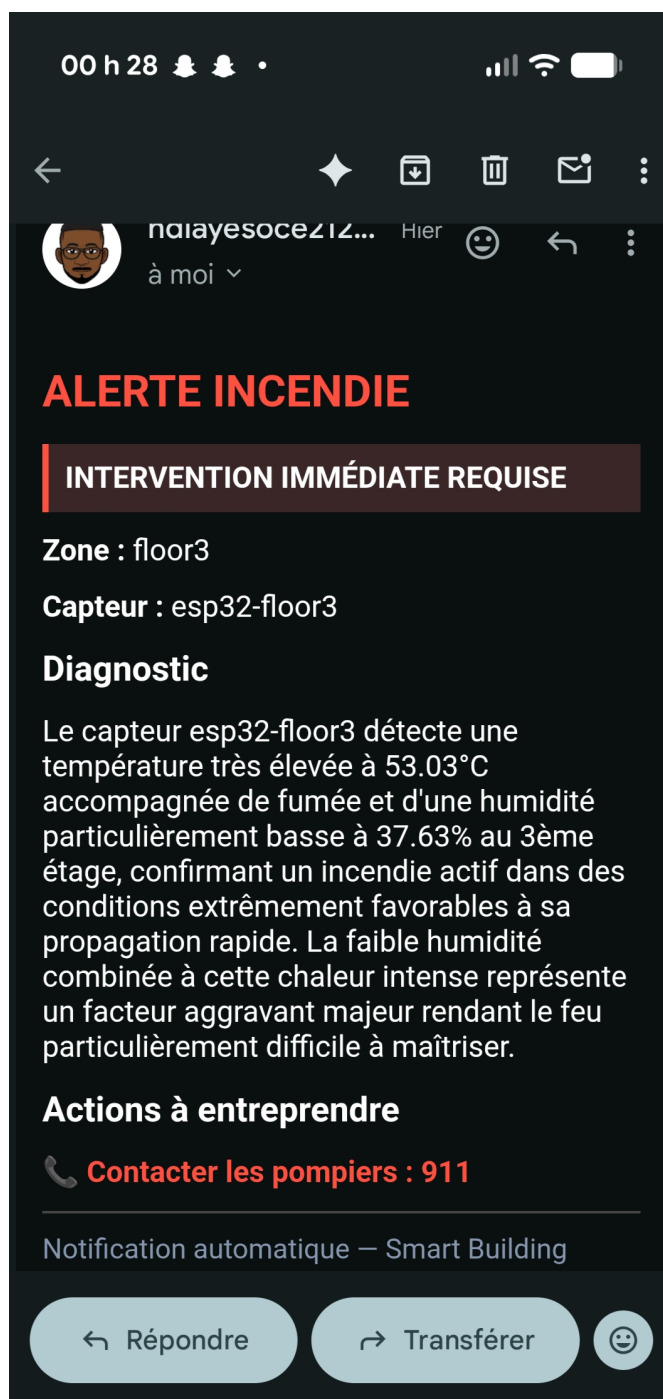


FIGURE 15 – Courriel d'alerte incendie reçu sur téléphone mobile lors du scénario S1

9 Conclusion

9.1 Objectifs atteints

Le projet a permis de réaliser une plateforme multi-agents complète et opérationnelle, couvrant l'intégralité du cycle de supervision intelligente : de la détection des anomalies jusqu'à la notification humaine. Les objectifs initialement fixés ont tous été atteints :

- Simulation d'un réseau IoT à quatre zones avec huit capteurs virtuels
- Détection temps réel des anomalies avec classification par sévérité
- Analyse par LLM avancé (Claude Sonnet 4.6 via PicoClaw)
- Orchestration automatisée des actions correctives via n8n
- Notifications par courriel avec contenu contextuel
- Dashboard temps réel avec création dynamique de zones

9.2 Apports techniques

Ce projet a permis de mettre en œuvre plusieurs technologies avancées et d'expérimenter leur intégration dans une architecture cohérente. Les principaux apports techniques sont :

- Maîtrise de l'architecture multi-agents avec chaînage par callbacks
- Intégration d'un LLM cloud dans un système temps réel via un framework agent léger
- Combinaison d'un pattern publish/subscribe MQTT avec des workflows HTTP
- Développement d'un dashboard temps réel avec gestion d'état persistant
- Discussion des compromis entre performance et souveraineté des données

9.3 Perspectives

Plusieurs pistes d'amélioration ont été identifiées pour prolonger le projet :

- Déploiement sur du matériel physique (ESP32 réels)
- Ajout de nouveaux types de capteurs (qualité d'eau, occupation, contrôle d'accès)
- Historisation des données pour des analyses prédictives
- Anonymisation des prompts pour un déploiement dans des contextes sensibles
- Intégration d'une base de données pour l'audit et la conformité

9.4 Impact et utilisation

La plateforme développée constitue une base solide pour un déploiement dans un contexte réel de smart building. L'architecture modulaire permet d'adapter facilement le système à différents types de bâtiments (bureaux, écoles, hôpitaux, entrepôts) en ajoutant ou modifiant les capteurs, les règles de sévérité et les workflows d'action. La combinaison d'un LLM avancé pour l'analyse contextuelle et de règles déterministes

pour la décision offre un bon équilibre entre intelligence et fiabilité, essentiel dans des applications où la sécurité des occupants est en jeu.